

Cloud Analytics Project Setup & ETL Cheat Sheet

PROJECT OVERVIEW

Goal:

Build an end-to-end cloud analytics platform using:

- CSV/Text Files
- Google Cloud Storage (GCS)
- Python ETL
- BigQuery
- Power BI

Final Architecture:

```
CSV Files
  ↓
Google Cloud Storage
  ↓
Python ETL Pipeline
  ↓
BigQuery
  ↓
Power BI
```

STEP 1 — CREATE PROJECT FOLDER

Example:

```
C:\olist-ecommerce-analytics
```

STEP 2 — CREATE PROJECT STRUCTURE

```
olist-ecommerce-analytics/
|
├─ data/
```

```
|   |   | raw/  
|   |   | processed/  
|   |   | archive/  
|  
|   | notebooks/  
|   | scripts/  
|   | sql/  
|   | screenshots/  
|   | powerbi/  
|   | credentials/  
|  
|   | README.md  
|   | requirements.txt  
|   | .gitignore
```

STEP 3 — OPEN TERMINAL INSIDE PROJECT FOLDER

Example:

```
cd C:\olist-ecommerce-analytics
```

STEP 4 — CREATE VIRTUAL ENVIRONMENT

```
python -m venv venv
```

STEP 5 — ACTIVATE VIRTUAL ENVIRONMENT

CMD

```
venv\Scripts\activate
```

PowerShell

```
.\env\Scripts\Activate.ps1
```

When activated you should see:

```
(venv)
```

STEP 6 — INSTALL REQUIRED PACKAGES

```
pip install pandas google-cloud-storage google-cloud-bigquery pandas-gbq  
pyarrow jupyter
```

STEP 7 — SAVE REQUIREMENTS

```
pip freeze > requirements.txt
```

STEP 8 — CREATE .gitignore FILE

Paste this:

```
venv/  
credentials/  
__pycache__/  
*.pyc  
.env  
.ipynb_checkpoints/
```

STEP 9 — CREATE GCP BUCKET

Recommended settings:

Setting	Value
Location Type	Region
Region	us-central1
Storage Class	Standard
Access Control	Uniform
Public Access Prevention	Enabled

Example bucket:

```
cdmx_mobility_insight2024
```

STEP 10 — CREATE BIGQUERY DATASET

Example:

```
rides_2024
```

STEP 11 — CREATE SERVICE ACCOUNT

Roles:

- Storage Admin
- BigQuery Admin

Download JSON key.

Store here:

```
credentials/olist-gcp.json
```

STEP 12 — SET GOOGLE CREDENTIALS

Option 1 — Environment Variable (CMD)

```
set GOOGLE_APPLICATION_CREDENTIALS=C:\olist-ecommerce-  
analytics\credentials\olist-gcp.json
```

Option 2 — Inside Python Script

```
import os  
  
os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = (  
    "credentials/olist-gcp.json"  
)
```

STEP 13 — START JUPYTER NOTEBOOK

```
jupyter notebook
```

STEP 14 — TEST GCS CONNECTION

```
from google.cloud import storage  
  
client = storage.Client()  
  
buckets = list(client.list_buckets())  
  
for bucket in buckets:  
    print(bucket.name)
```

STEP 15 — UPLOAD FILE TO GCS

```
from google.cloud import storage

PROJECT_ID = "cdmxmobility"
BUCKET_NAME = "cdmx_mobility_insight2024"

FILE_PATH = "data/processed/final_orders.csv"
DEST_BLOB_NAME = "final_orders.csv"

storage_client = storage.Client(project=PROJECT_ID)

bucket = storage_client.bucket(BUCKET_NAME)
blob = bucket.blob(DEST_BLOB_NAME)

blob.upload_from_filename(FILE_PATH)

print("File uploaded successfully")
```

STEP 16 — LOAD CSV INTO BIGQUERY

```
from google.cloud import bigquery

PROJECT_ID = "cdmxmobility"
DATASET_ID = "rides_2024"
TABLE_ID = "final_orders"
BUCKET_NAME = "cdmx_mobility_insight2024"

uri = f"gs://{BUCKET_NAME}/final_orders.csv"

client = bigquery.Client(project=PROJECT_ID)

table_ref = f"{PROJECT_ID}.{DATASET_ID}.{TABLE_ID}"

job_config = bigquery.LoadJobConfig(
    source_format=bigquery.SourceFormat.CSV,
    skip_leading_rows=1,
    autodetect=True,
    write_disposition="WRITE_TRUNCATE"
)

load_job = client.load_table_from_uri(
```

```
        uri,  
        table_ref,  
        job_config=job_config  
    )  
  
    load_job.result()  
  
    print("Data loaded successfully")
```

STEP 17 — CREATE RFM TABLE

```
import pandas as pd  
  
final_orders = pd.read_csv(  
    "data/processed/final_orders.csv"  
)  
  
final_orders['order_purchase_timestamp'] = pd.to_datetime(  
    final_orders['order_purchase_timestamp']  
)  
  
snapshot_date = (  
    final_orders['order_purchase_timestamp'].max()  
    + pd.Timedelta(days=1)  
)  
  
rfm = final_orders.groupby('customer_unique_id').agg({  
    'order_purchase_timestamp': lambda x: (snapshot_date - x.max()).days,  
    'order_id': 'nunique',  
    'payment_value': 'sum'  
}).reset_index()  
  
rfm.columns = [  
    'customer_unique_id',  
    'Recency',  
    'Frequency',  
    'Monetary'  
]  
  
print(rfm.head())
```

STEP 18 — EXPORT RFM TABLE

```
rfm.to_csv(  
    'data/processed/final_orders_rfm.csv',  
    index=False  
)
```

STEP 19 — LOAD RFM TABLE INTO BIGQUERY

```
TABLE_ID = "final_orders_rfm"  
  
uri = f"gs://{BUCKET_NAME}/final_orders_rfm.csv"
```

Repeat the same BigQuery loading process.

STEP 20 — CONNECT POWER BI TO BIGQUERY

In Power BI:

```
Get Data  
↓  
Google BigQuery
```

Select:

```
cdmxmobility
```

Then:

```
rides_2024
```

Import:

- final_orders
- final_orders_rfm

STEP 21 — BUILD DASHBOARDS

Recommended pages:

1. Executive Overview

Focus:

- revenue
 - customers
 - orders
 - sales trends
-

2. Logistics Dashboard

Focus:

- delivery delays
 - shipping performance
 - operational efficiency
-

3. RFM Dashboard

Focus:

- customer segmentation
 - retention
 - loyalty
 - customer value
-

STEP 22 — SAVE POWER BI FILE

Example:

`powerbi/olist_ecommerce_analytics.pbix`

STEP 23 — GIT COMMANDS

Initialize Git

```
git init
```

Add Files

```
git add .
```

Commit

```
git commit -m "Initial commit"
```

Connect GitHub Repo

```
git remote add origin YOUR_REPO_URL
```

Push to GitHub

```
git branch -M main
```

```
git push -u origin main
```

STEP 24 — AUTOMATION (NEXT LEVEL)

Future automation workflow:

```
CSV Uploaded to GCS
```

```
↓
```

Cloud Function Triggered



Python ETL Runs



BigQuery Updated



Power BI Refreshes

Tools:

- Cloud Functions
- Cloud Scheduler
- Cloud Run

FINAL PORTFOLIO SKILLS

This project demonstrates:

- Python ETL
- Cloud Storage
- BigQuery
- SQL Analytics
- Power BI
- Customer Analytics
- Logistics Analytics
- Cloud Architecture
- Automation Workflow
- GitHub Project Structuring

RECOMMENDED PROJECT TITLE

Cloud-Based E-Commerce Analytics Platform

or

End-to-End Cloud Analytics Pipeline with BigQuery & Power BI

FINAL CAREER VALUE

This project is suitable for:

- Data Analyst Portfolio
- BI Analyst Portfolio
- Analytics Engineer Portfolio
- Junior Data Engineer Portfolio

because it combines:

- analytics
- engineering
- cloud
- automation
- business intelligence
- customer analytics